React Framework

Ehsan Khamespanah

Spring 2026

What is React

- A JavaScript library for building user interfaces, especially single-page applications
  - Developed by Meta Platforms
  - Uses component-based architecture
  - Efficient updates with Virtual DOM
- Why use React?
  - Reusable components
  - Fast rendering
  - Strong community

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Hello World</title>
    <script src="js/react.development.js"></script>
    <script src="js/react-dom.development.js"></script>
    <script src="js/babel.min.js"></script>
  </head>

  <body>
    <div id="root"></div>
    <script type="text/babel" src="jsx/main.jsx"></script>
 </body>

</html>
```

```
const root = ReactDOM.createRoot(document.getElementById("root"));
const element = (
    <h1>Hello, world2!</h1>
);
root.render(element);
```

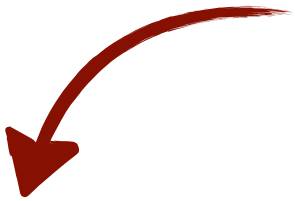
Introducing JSX

```
function formatName(user){
    return user.firstName + ' ' + user.lastName;
}

const user = {
    firstName: 'Harper',
    lastName: 'Perez'
};

const element = (
    <h1>
        Hello, {formatName(user)}!
    </h1>
);

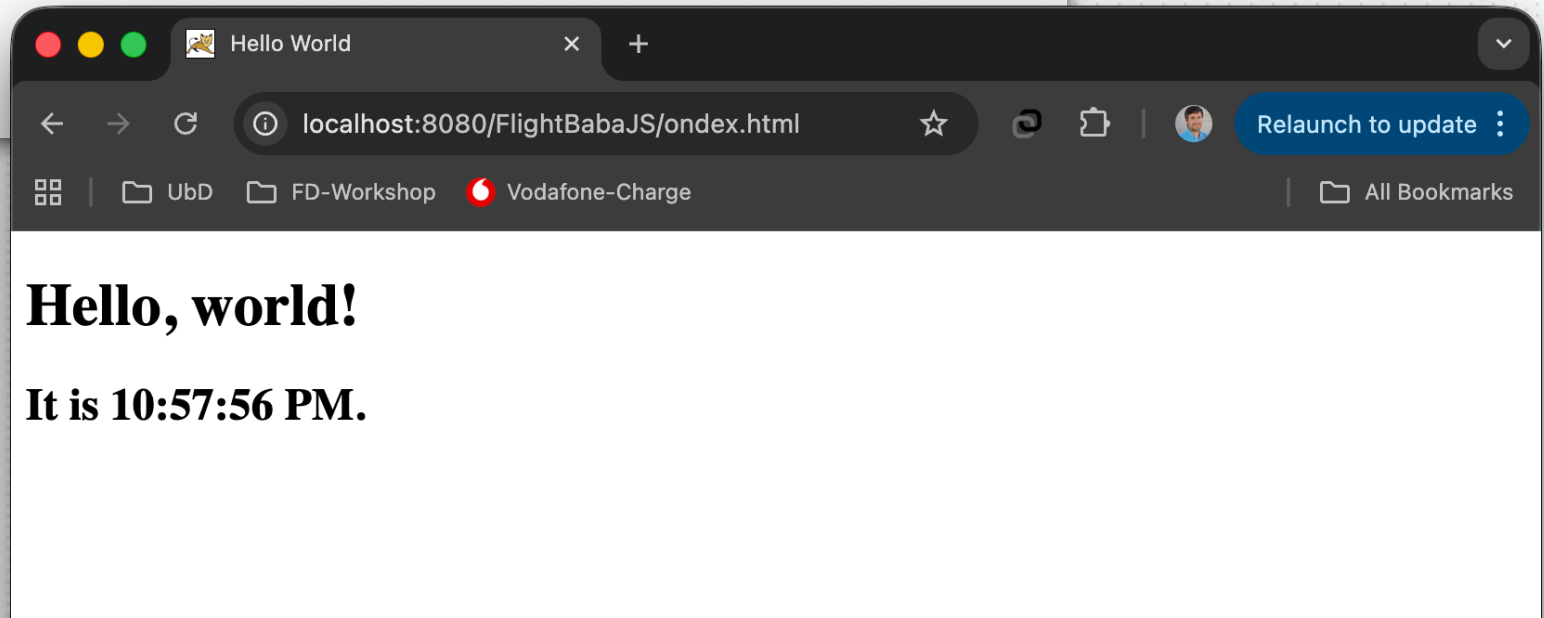
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(element);
```



JS code to provide value

Updating Rendered Elements

```
function tick() {
    const element = (
        <div>
            <h1>Hello, world!</h1>
            <h2>It is {new Date().toLocaleTimeString()}.</h2>
        </div>
    );
    const root = ReactDOM.createRoot(document.getElementById("root"));
    root.render(element);
}
setInterval(tick, 1000);
```



React Components

- Components let you split the UI into independent, reusable pieces, and think about each piece in isolation.




```
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}
const element = <Welcome name=“Sara"/>;
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(element);
```

Composing Components

```
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}
function App() {
  return (
    <div>
      <Welcome name="Sara" />
      <Welcome name="Cahal" />
      <Welcome name="Edite" />
    </div>
  );
}
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);
```

Extracting Components

```
function Comment(props) {
  return (
    <div className="Comment">
      <div className="UserInfo">
        <img className="Avatar" src={props.author.avatarUrl}
            alt={props.author.name}
        />
        <div className="UserInfo-name">
          {props.author.name}
        </div>
      </div>
      <div className="Comment-text">
        {props.text}
      </div>
      <div className="Comment-date">
        {formatDate(props.date)}
      </div>
    </div>
  );
}
```

Extracting Components

```
function Comment(props) {
  return (
    <div className="Comment">
      <div className="UserInfo">
        <img className="Avatar" src={props.author.avatarUrl}
           alt={props.author.name}
        />
        <div className="UserInfo-name">
          {props.author.name}
        </div>
      </div>
      <div className="Comment-text">
        {props.text}
      </div>
      <div className="Comment-date">
        {formatDate(props.date)}
      </div>
    </div>
  );
}
```

Extracting Components

```
function Comment(props) {
  return (
    <div className="Comment">
      <div className="UserInfo">
        <img className="Avatar" src={props.author.avatarUrl}
            alt={props.author.name}
        />
        <div className="UserInfo-name">
          {props.author.name}
        </div>
      </div>
      <div className="Comment-text">
        {props.text}
      </div>
      <div className="Comment-date">
        {formatDate(props.date)}
      </div>
    </div>
  );
}
```

```
function Avatar(props) {
  return (
    <img className="Avatar"
      src={props.user.avatarUrl}
      alt={props.user.name}
    />
  );
}
```

Extracting Components

```
function Comment (props) {
  return (
    <div className="Comment">
      <div className="UserInfo">
        <Avatar user={props.author} />
        <div className="UserInfo-name">
          {props.author.name}
        </div>
      </div>
      <div className="Comment-text">
        {props.text}
      </div>
      <div className="Comment-date">
        {formatDate (props.date)}
      </div>
    </div>
  );
}
```

Extracting Components

```
function Comment (props) {
  return (
    <div className="Comment">
      <div className="UserInfo">
        <Avatar user={props.author} />
        <div className="UserInfo-name">
          {props.author.name}
        </div>
      </div>
      <div className="Comment-text">
        {props.text}
      </div>
      <div className="Comment-date">
        {formatDate (props.date)}
      </div>
    </div>
  );
}
```

Extracting Components

```
function Comment (props) {
  return (
    <div className="Comment">
      <div className="UserInfo">
        <Avatar user={props.author} />
        <div className="UserInfo-name">
          {props.author.name}
        </div>
      </div>
      <div className="Comment-text">
        {props.text}
      </div>
      <div className="Comment-date">
        {formatDate (props.date)}
      </div>
    </div>
  );
}
```

```
function UserInfo(props) {
  return(
    <div className="UserInfo">
      <Avatar user={props.user} />
      <div className="UserInfo-name">
        {props.user.name}
      </div>
    </div>
  );
}
```

Extracting Components

```
function Comment (props) {
  return (
    <div className="Comment">
      <UserInfo user={props.author} />
      <div className="Comment-text">
        {props.text}
      </div>
      <div className="Comment-date">
        {formatDate (props.date)}
      </div>
    </div>
  );
}
```

Events

- Event handlers receive a React event object. It is also sometimes known as a “synthetic event”
- It conforms to the same standard as the underlying DOM events

```
function ClickExample() {
  function handleClick(e) {
    console.log(e);
    alert(“Clicked!");
  }
  return <button onClick={e =>
    handleClick(e)}>Click Me!</button>;
}
```

- React event objects implement some methods:
  - preventDefault(): Prevents the default browser action for the event
  - stopPropagation(): Stops the event propagation through the React tree

States of Components

- We need a set of variables to store the state of a component
- In React, hooks are special functions that let you “hook into” React features (like state and lifecycle) inside functional components
- Hooks allow you to:
  - Store and update state
  - Run side effects (like API calls)
  - Access lifecycle behavior (mount, update, unmount)
  - Share logic between components

A Toggling Button - Handling State and Event

```
function ToggleButton() {
  // State hook
  const [isOn, setIsOn] = React.useState(false);

  function handleClick() {
    // Update state with the setter function
    setIsOn(!isOn);
  }

  return (
    <button onClick={handleClick}>
      {isOn ? "ON" : "OFF"}
    </button>
  );
}

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<ToggleButton />);
```

Core Idea Behind Hooks (Simplified)

```
let hooks = [];
let currentHook = 0;

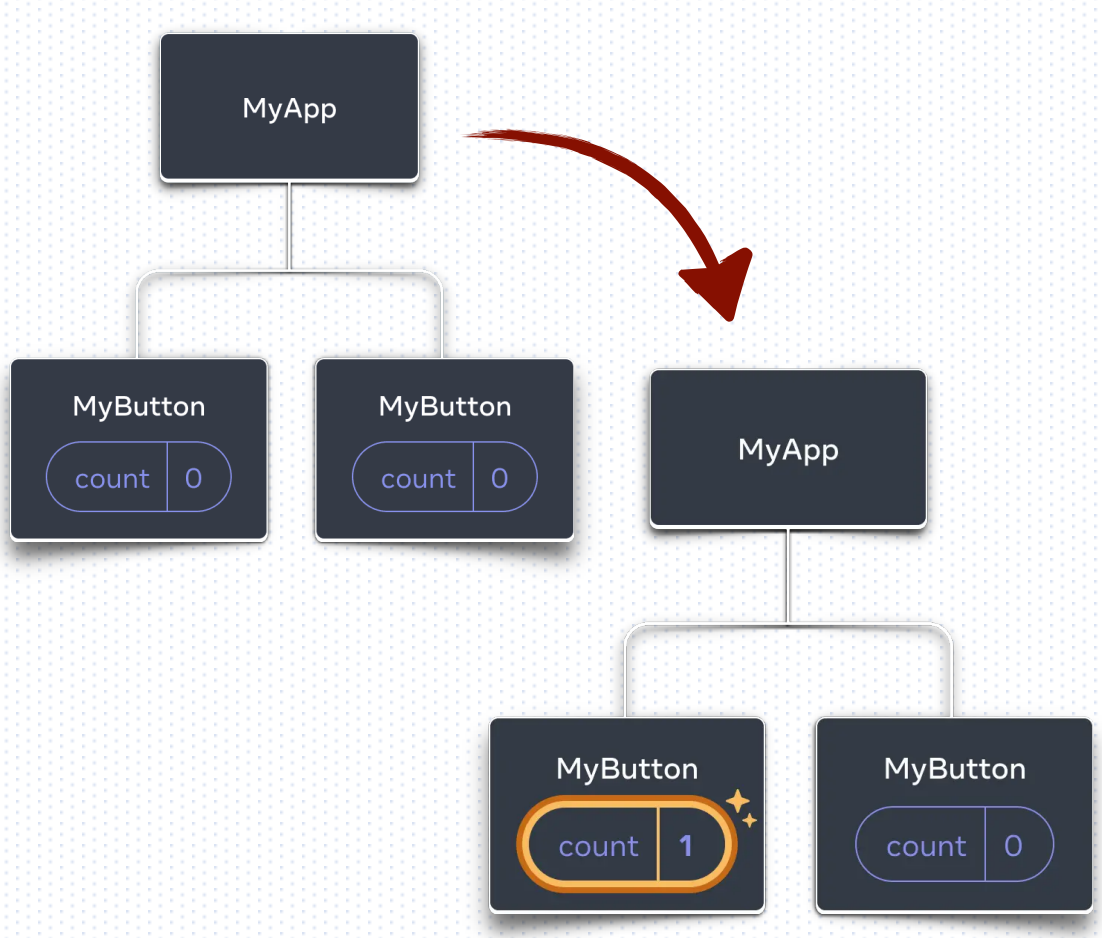
function useState(initialValue) {
  const hookIndex = currentHook;

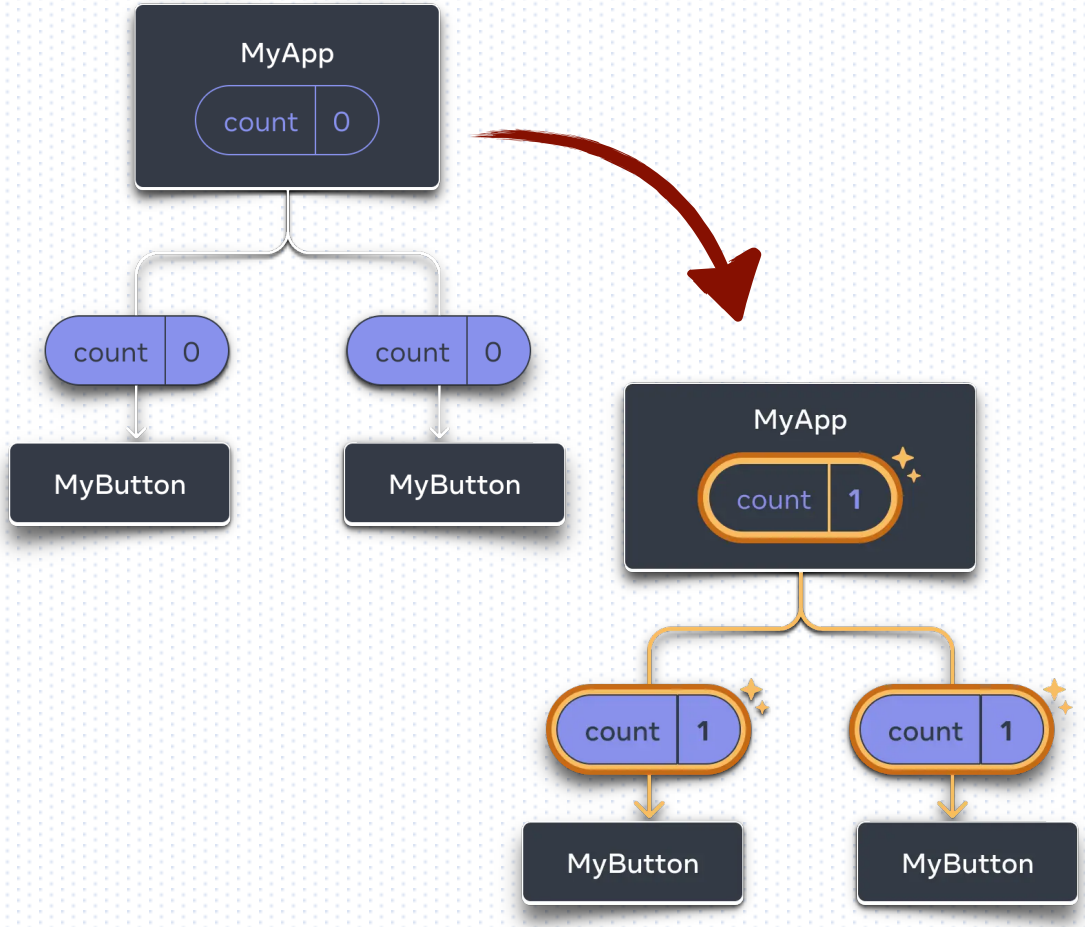
  if (!hooks[hookIndex]) {
    hooks[hookIndex] = initialValue;
  }

  function setState(newValue) {
    hooks[hookIndex] = newValue;
    reRender();
  }

  currentHook++;
  return [hooks[hookIndex], setState];
}
```

Sharing Data Between Components





Side Effects

- useEffect is one of the most important React Hooks
- It allows performing side effects in function components
- Side effect: operations that interact with something outside the component
  - Fetching data from an API
  - Updating the DOM manually
  - Setting timers (setTimeout, setInterval)
  - Adding event listeners
  - Accessing browser APIs like localStorage
  - Subscribing to WebSocket connections

How to Consider Side Effects

- Basic Syntax

```
useEffect(() => {
  // Code to run after rendering

  return () => {
    // Cleanup before next effect or component unmount
  };
}, [dependencies]);
```

This part runs after React renders the component.

Remove event listeners,
Clear timers,
Cancel subscriptions,
Abort fetch requests

Controls when the effect runs

How React Executes useEffect

- React renders the component
- React updates the DOM
- React runs the effect
- If dependencies change:
  - React runs the cleanup part from the previous effect
  - React runs the new effect
- When the component unmounts:
  - React runs cleanup

An Example

```
function Counter() {
  const [count, setCount] = React.useState(0);

  document.title = `Count: ${count}`; // Bad: runs during rendering

  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  );
}
```

An Example

```
function Counter() {
  const [count, setCount] = React.useState(0);

  document.title = `Count: ${count}`; // Bad: runs during rendering

  return (
    <button
      {count
    </button
  );
}
```

```
function Counter() {
  const [count, setCount] = React.useState(0);

  React.useEffect(() => {
    document.title = `Count: ${count}`;
  });

  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  );
}
```

Three Main Dependency Patterns

- No Dependency Array

```
useEffect(() => {
  console.log("Runs after every render");
});
```

- Runs in the initial render
- Also in every re-render

Three Main Dependency Patterns

- Empty Dependency Array []

```
useEffect(() => {
  console.log("Runs only once");
}, []);
```

- Runs only after the first render (mount)
- Equivalent to componentDidMount in the deprecated way of defining components in React

Three Main Dependency Patterns

- Specific Dependencies

```
useEffect(() => {
  console.log("Runs when count changes");
}, [count]);
```

- Run on the first render
- Also, whenever “count” changes

Fetch Data from API

- It most be after the first render

```
function FlightList() {
  const [flights, setFlights] = React.useState([]);

  React.useEffect(() => {
    fetch("https://http://localhost:8080/FlightBabaJS/flights")
      .then(response => response.json())
      .then(data => setFlights(data));
  }, []);

  return (
    <ul>
      {flights.map(flight => (
        <li key={flight.id}>{flight.destination}</li>
      ))}
    </ul>
  );
}
```

Watching State Changes

- Do something in the background based on the typed query value

```
function Search() {
  const [query, setQuery] = React.useState("");

  React.useEffect(() => {
    console.log("Searching for:", query);
  }, [query]);

  return (
    <input
      value={query}
      onChange={e => setQuery(e.target.value)}
    />
  );
}
```

Cleanup Function Example

- When the component unmounts, the interval is cleared

```
function Timer() {
  React.useEffect(() => {
    const id = setInterval(() => {
      console.log("Tick");
    }, 1000);

    return () => {
      clearInterval(id);
    };
  }, []);

  return <h1>Timer Running</h1>;
}
```

Cleanup Before Re-Running

- If the value of id changes:
  - Cleanup the old subscription
  - Start a new subscription

```
function Example({ id }) {
  React.useEffect(() => {
    console.log("Subscribe to", id);

    return () => {
      console.log("Unsubscribe from", id);
    };
  }, [id]);
}
```

Window Resize Listener Example

```
function WindowWidth() {
  const [width, setWidth] = React.useState(window.innerWidth);

  React.useEffect(() => {
    function handleResize() {
      setWidth(window.innerWidth);
    }

    window.addEventListener("resize", handleResize);

    return () => {
      window.removeEventListener("resize", handleResize);
    };
  }, []);

  return <h1>Width: {width}</h1>;
}
```

You can use several useEffect hooks

```
function Example() {
  React.useEffect(() => {
    console.log("Mount effect");
  }, []);

  React.useEffect(() => {
    console.log("Count changed");
  }, [count]);
}
```

Timeline Example

```
function Demo() {
  const [count, setCount] = React.useState(0);

  React.useEffect(() => {
    console.log("Effect runs:", count);

    return () => {
      console.log("Cleanup:", count);
    };
  }, [count]);

  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  );
}
```

```
Initial render:
Effect runs: 0

Click button:
Cleanup: 0
Effect runs: 1

Click again:
Cleanup: 1
Effect runs: 2
```

Input and Forms

- Storing the state of a form in state variables and calling services using these state variables

```
function Form() {
  const [text, setText] = React.useState("");

  return (
    <>
      <input
        value={text}
        onChange={(e) => setText(e.target.value)}
      />
      <p>You typed: {text}</p>
    </>
  );
}
```

More About JSX and React

- Lists

```
const items = ["Milk", "Egg", “Bread"];

function List() {
  return (
    <ul>
      {items.map((item, index) => (
        <li key={index}>{item}</li>
      ))}
    </ul>
  );
}
```

- Conditional Rendering

```
function Status({ isLoggedIn }) {
  return isLoggedIn?
    <h1>Welcome</h1> :
    <h1>Please log in</h1>;
}
```